



Jogo Web de Gestao de Recursos

Projeto Final - Desenvolvimento de Aplicacoes Web
Escola Superior de Ciencias Empresariais - IPS
Ano Letivo 2025/2026

Bernardo Pinheiro, Ines Medeiros, Ricardo Janz

1. Introdução

O presente relatório descreve o projeto final da unidade curricular Desenvolvimento de Aplicações Web (DAW), lecionada na Escola Superior de Ciências Empresariais do Instituto Politécnico de Setúbal (IPS/ESCE) no ano letivo 2025/2026.

O projeto consiste no desenvolvimento de um jogo web de gestão de recursos, com temática inspirada no universo Minecraft. O jogador controla a personagem Steve num cenário 2D, onde pode cortar árvores para obter madeira, minerar pedras, construir edifícios (Mesa de Trabalho, Fornalha e Mineradora de Diamantes), iniciar tarefas temporizadas e recolher recompensas. O jogo inclui ainda um sistema de evolução do machado, uma loja de skins e um leaderboard global.

O objetivo principal foi consolidar os conhecimentos adquiridos ao longo das aulas práticas (laboratórios 1 a 8), nomeadamente nos domínios da programação JavaScript (manipulação do DOM, estruturas de dados, persistência), programação servidor em Python/Flask (padrão MVC, autenticação, base de dados) e integração frontend-backend através da Fetch API.

2. Arquitetura da Solução

2.1 Visão Geral

A aplicação segue o padrão MVC (Model-View-Controller), com uma separação clara entre a lógica de negócio, a apresentação e os dados. O backend é implementado em Flask (Python) e o frontend em HTML5, CSS3 e JavaScript vanilla, sem dependências de frameworks JavaScript externas.

A comunicação entre frontend e backend é feita através de requisições HTTP assíncronas utilizando a Fetch API, com dados em formato JSON. As páginas HTML são renderizadas no servidor usando o motor de templates Jinja2.

2.2 Estrutura de Diretórios

```
minecraft2d/
├── server.py           # Ponto de entrada
├── auth.py             # Blueprint autenticação
├── game.py             # Blueprint do jogo
├── settings.py         # Configurações
├── extensions.py       # LoginManager
├── requirements.txt    # Dependências
├── base_dados.sql      # Esquema SQL
├── api_documentacao.md # Documentação API
├── README.md           # Instruções
├── models/
│   ├── __init__.py    # Modelos + user_loader
│   ├── database.py    # Database + modelos
│   └── minecraft2d.sqlite # Base de dados
├── templates/         # 6 templates Jinja2
├── static/
│   ├── css/           # 10 folhas modulares
│   ├── js/            # game.js, theme.js
│   └── img/           # Recursos gráficos
```

2.3 Tecnologias Utilizadas

- Backend: Flask, Flask-Login, SQLite 3, passlib (pbkdf2_sha256)
- Frontend: HTML5 semântico, CSS3 (Flexbox, Grid, custom properties), JavaScript vanilla
- Autenticação: Sessões baseadas em cookies com Flask-Login
- Base de Dados: SQLite 3 com row factory (sqlite3.Row)

- Controlo de Versao: Git + GitHub

3. Base de Dados - Minecraft2D

O projeto usa SQLite via sqlite3 (Python puro, sem ORM). O ficheiro esta em models/minecraft2d.sqlite. A classe Database (models/database.py) e o data access layer que faz todas as queries.

3.1 Tabelas (6 no total)

1. users

Campos: id, username, email, password_hash, wood, stone, iron, diamonds, has_axe, axe_level, skin, created_at

Conta dos jogadores. Criada no /register (auth.py), lida/atualizada em cada acao de jogo (cortar madeira, minerar pedra, construir, recolher recompensas). O password_hash usa pbkdf2_sha256 (passlib). has_axe/axe_level sao para o machado da Mesa de Trabalho. diamonds e obtido como recompensa da Mineradora de Diamantes.

2. building_slots

Campos: id, user_id (FK->users), slot_number (1-4), building_type, state, action_type, started_at, ready_at, created_at

4 slots de construcao por jogador. Cada slot tem uma maquina de estados: empty -> building -> ready -> working -> collectable -> ready. Criados automaticamente no registo. Atualizados nas rotas /api/build, /api/task/start, /api/task/collect (game.py). Permitem construcoes e tarefas temporizadas sem cron jobs (compara ready_at com o relógio do servidor).

3. action_logs

Campos: id, user_id (FK->users), message, created_at

Historico de acoes de cada jogador (ex: "Arvore cortada: +4 madeira"). Inserido em cada acao de jogo (cortar, minerar, construir, etc.). Mostrado na dashboard (ultimas 8 acoes).

4. trees

Campos: id, column (2, 8, 15), chopped_at, removed_at, created_at

Arvores no mapa, com sistema de cooldown de 10s (RESOURCE_RESPAWN_SECONDS). Lidas/atualizadas na rota /api/chop. Se chopped_at ja passou do cooldown, a arvore fica disponivel novamente.

5. stones

Campos: id, column (1, 4, 7, 10), mined_at, removed_at, created_at

Pedras no mapa, mesmo sistema de cooldown das arvores. Lidas/atualizadas na rota /api/mine-stone. Logica identica as arvores.

6. buildings

Campos: key (PK), name, cost_wood, cost_stone, cost_iron, construction_seconds, task_name, task_seconds, reward_wood, reward_stone, reward_iron, reward_diamond, description

Catalogo de construcoes disponiveis (dados estaticos seedados na criacao da BD). 3 edificios:

- cabana -> Mesa de Trabalho (fabrica/upgrade do machado)
- forja -> Fornalha (funde minerio -> ferro)
- mina -> Mineradora de Diamantes (extrai diamantes; custo: 10 madeira + 10 pedra + 20 ferro; constroi em 30s; tarefa: 30s; recompensa: 1 diamante)

Lido ao mostrar a dashboard e ao validar construcoes/tarefas. Nunca e escrito durante o jogo.

3.2 Interacoes Principais (Fluxo de Dados)

Registo -> INSERT users + 4 building_slots 'empty' + action_log

Login	-> SELECT users (valida password)
Dashboard	-> SELECT users + building_slots + action_logs + trees + stones + buildings
Cortar arvore	-> UPDATE trees (chopped_at) + UPDATE users (wood+4) + INSERT action_log
Minerar pedra	-> UPDATE stones (mined_at) + UPDATE users (stone+2) + INSERT action_log
Construir	-> UPDATE users (deduz custo) + UPDATE slot (estado 'building', ready_at)
Recolher tarefa	-> UPDATE users (soma recompensa: madeira, pedra, ferro ou diamante; ou da machado) + UPDATE slot (volta a 'ready')
Leaderboard	-> SELECT users ORDER BY (wood+stone+iron+diamonds) DESC LIMIT 10

3.3 Seguranca

Todas as queries usam ? como placeholder (protegido contra SQL injection). As ligacoes sao abertas com `connection.row_factory = sqlite3.Row` para aceder a colunas por nome.

4. Backend (Flask)

4.1 Autenticacao

O modulo auth.py implementa as rotas de autenticacao: registo (/register), login (/login) e logout (/logout). Utiliza Flask-Login para gestao de sessoes e passlib (pbkdf2_sha256) para hashing de passwords. As validacoes incluem: campos obrigatorios, comprimento minimo da password (4 caracteres) e unicidade de username/email.

O registo cria automaticamente 4 slots de construcao vazios para o novo utilizador e regista a primeira entrada no historico de acoes.

4.2 Rotas do Jogo

O modulo game.py (Blueprint game) implementa as seguintes rotas:

- GET / | /dashboard - Pagina principal do jogo (renderizada com Jinja2)
- GET /api/state - Estado completo do jogo em JSON (user, slots, logs, arvores, pedras)
- POST /api/build/<id> - Inicia construcao num slot vazio
- POST /api/task/<id>/start - Inicia tarefa num slot pronto
- POST /api/task/<id>/collect - Recolhe recompensa de tarefa concluida
- POST /api/chop - Corta arvore (recebe coluna no JSON body)
- POST /api/mine-stone - Mina pedra (recebe coluna no JSON body)
- POST /api/slot/<id>/remove - Remove construcao de um slot
- POST /api/inventory/remove - Remove recursos do inventario manualmente
- GET /api/leaderboard - Top 10 jogadores (JSON)
- GET /skins, /api/skins, POST /api/skin/select - Sistema de personalizacao de skins

4.3 Maquina de Estados dos Slots

Cada slot de construcao segue o ciclo: empty -> building -> ready -> working -> collectable -> ready. Os timestamps (started_at, ready_at) sao guardados em ISO datetime e comparados com o tempo atual do servidor a cada requisicao, eliminando a necessidade de cron jobs ou processos em background.

4.4 Sistema do Machado

A Mesa de Trabalho permite fabricar e fazer upgrade do machado. O nivel 1 duplica a madeira obtida por arvore. Niveis superiores aumentam progressivamente o rendimento. Os custos de upgrade escalam com o nivel: niveis 1-2 usam madeira e pedra, nivel 3 adiciona ferro, niveis 4+ usam diamantes.

5. Frontend

5.1 Estrutura HTML e Templates

O frontend é composto por 6 templates Jinja2 que estendem um layout base (base.html). O layout base inclui uma barra de navegação com ligações para Dashboard, Skins, Leaderboard e Logout, além de um botão de alternância entre modo claro e escuro. As páginas de autenticação (login, register) seguem um design Minecraft temático.

5.2 CSS Modular

O estilo é organizado em 10 ficheiros CSS modulares com @import:

- variables.css: propriedades personalizadas (cores, sombras, espaçamentos)
- typography.css: font-face, body font-stack, estilos base
- layout.css: topbar, navegação, auth pages, botões, responsividade
- game.css: dashboard, cena de jogo, inventário, slots, seletor de construções
- leaderboard.css: grelha de ranking, itens, badges de podium
- skins.css: grelha de skins, cartões, botões de seleção
- theme.css: modo escuro, cena dia/noite, nuvens, estrelas, pirilampos

5.3 JavaScript (game.js)

O ficheiro game.js (884 linhas) implementa toda a lógica do lado do cliente: renderização da cena 2D (grelha, árvores, pedras, Steve), controlo de movimento (setas direcionais, teclas A/D, botões na interface), interação com recursos (clique em árvores/pedras), Fetch API para todas as requisições ao backend, atualização em tempo real do inventário e slots com indicadores de progresso, e sistema de notificações toast para feedback ao utilizador.

5.4 Responsividade

A interface é responsiva com 3 breakpoints: 980px (tablet), 720px e 430px (mobile). Utiliza Flexbox e CSS Grid para layout, garantindo que o jogo é jogável em qualquer dispositivo.

6. Mecanica do Jogo

6.1 Recursos

O jogo possui 4 recursos: madeira (wood), pedra (stone), ferro (iron) e diamantes (diamonds). Cada novo jogador começa com 26 de madeira e 26 de pedra. O limite maximo de cada recurso e 64 unidades.

6.2 Extracao de Recursos

O mapa contem 3 arvores (colunas 2, 8, 15) e 4 pedras (colunas 1, 4, 7, 10). Cada extracao rende 4 madeira ou 2 pedra, respetivamente. Apos extracao, o recurso fica indisponivel durante 10 segundos (cooldown).

Se o jogador possuir um machado, a madeira obtida e multiplicada por $(axe_level + 1)$. Por exemplo, com machado nivel 3, cada arvore rende $4 \times 4 = 16$ madeira.

6.3 Construcoes e Tarefas

O jogador dispoe de 4 slots de construcao. Existem 3 tipos de construcao:

- Mesa de Trabalho (cabana): Permite fabricar o machado inicial (custo: 15 madeira + 5 pedra) e fazer upgrades progressivos.
- Fornalha (forja): Funde minerio em lingotes de ferro. Custo: 10 madeira + 15 pedra. Cada tarefa concluida produz 1 ferro.
- Mineradora de Diamantes (mina): Extrai diamantes. Custo: 10 madeira + 10 pedra + 20 ferro. Cada tarefa concluida produz 1 diamante.

6.4 Ciclo de Vida de um Slot

1. Slot vazio (empty) -> jogador paga custo e inicia construcao
2. Construcao em progresso (building) -> espera pelo temporizador
3. Construcao pronta (ready) -> jogador pode iniciar uma tarefa
4. Tarefa em progresso (working) -> espera pelo temporizador
5. Tarefa concluida (collectable) -> jogador recolhe a recompensa
6. Slot volta ao estado ready para nova tarefa

6.5 Skins

O jogo inclui 5 skins: Steve (default), Steve Dourado (filtro sepia), Steve das Sombras (escuro), Steve Bronzeado (tons quentes) e Steve Rosa (tom roxo). As skins sao aplicadas via filtros CSS sobre a mesma imagem base, sem necessidade de recursos graficos adicionais.

6.6 Leaderboard

O leaderboard apresenta o top 10 de jogadores ordenados por pontuacao total (soma de todos os recursos). As tres primeiras posicoes exibem medalhas (ouro, prata, bronze). A lista e atualizada automaticamente a cada 5 segundos no frontend via Fetch API.

7. Documentacao da API

A API do backend expoe 13 endpoints. Todos requerem autenticacao via Flask-Login (cookie de sessao), exceto /login e /register. A documentacao completa com exemplos de requests e responses encontra-se no ficheiro api_documentacao.md.

Resumo dos endpoints:

Metodo	Rota	Descricao
GET	/login	Formulario de login
POST	/login	Autenticar utilizador
GET	/register	Formulario de registo
POST	/register	Criar nova conta
GET POST	/logout	Terminar sessao
GET	/ /dashboard	Pagina principal do jogo
GET	/api/state	Estado completo do jogo (JSON)
POST	/api/build/<slot_id>	Iniciar construcao
POST	/api/task/<slot_id>/start	Iniciar tarefa
POST	/api/task/<slot_id>/collect	Recolher recompensa
POST	/api/chop	Cortar arvore
POST	/api/mine-stone	Minerar pedra
POST	/api/slot/<slot_id>/remove	Remover construcao
POST	/api/inventory/remove	Remover recursos manualmente
GET	/leaderboard	Pagina de classificacao
GET	/api/leaderboard	Top 10 jogadores (JSON)
GET	/skins	Loja de skins
GET	/api/skins	Lista de skins (JSON)
POST	/api/skin/select	Selecionar skin

8. Instalacao e Execucao

Requisitos minimos: Python 3.12+, pip, navegador atual.

```
# 1. Clonar repositorio e entrar na pasta
git clone <url> && cd minecraft2d

# 2. Criar e ativar ambiente virtual
python3 -m venv .venv
source .venv/bin/activate

# 3. Instalar dependencias
pip install -r requirements.txt

# 4. Executar o servidor
python server.py

# 5. Abrir no navegador
http://127.0.0.1:8001
```


9. Conclusão

O projeto Minecraft 2D foi desenvolvido com sucesso, cumprindo todos os requisitos obrigatórios definidos no enunciado e diversas funcionalidades extra:

- Sistema de autenticação completo (registro, login, logout) com sessões persistidas
- 4 tipos de recursos (madeira, pedra, ferro, diamantes) com limites
- 3 tipos de construções distintas com custos, tempos e funcionalidades
- 4 slots de construção por jogador com máquina de estados (5 estados)
- Mecânica de tempo sem cron jobs (comparação de timestamps)
- Sistema de tarefas com recompensas recolhidas manualmente
- Frontend responsivo (desktop, tablet, mobile)
- Comunicação frontend-backend via Fetch API (JSON)
- Leaderboard global (top 10) com atualização em tempo real
- Loja de skins com 5 opções e filtros CSS
- Sistema de evolução do machado com 4+ níveis
- Modo escuro com transição dia/noite e efeitos visuais
- Base de dados SQLite com esquema completo e dados iniciais (seed)
- Código modular com Blueprints e separação MVC

Para além dos requisitos mínimos, o projeto inclui skins com filtros CSS, sistema de machado com upgrades progressivos, cena 2D interativa com animações, tema claro/escuro com efeitos visuais (nuvens, estrelas, pirilampos), e uma interface polida e consistente com o tema Minecraft.

O código fonte encontra-se organizado de forma modular, com separação clara entre modelos, rotas e templates, facilitando a manutenção e evolução futura do projeto.

10. Referências

- Flask Documentation (2025). <https://flask.palletsprojects.com/>
- MDN Web Docs (2025). Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- MDN Web Docs (2025). HTML. <https://developer.mozilla.org/en-US/docs/Web/HTML>
- Flask-Login Documentation. <https://flask-login.readthedocs.io/>
- passlib Documentation. <https://passlib.readthedocs.io/>
- OWASP (2025). Top 10 Web Application Security Risks. <https://owasp.org/www-project-top-ten/>